ATLAS: Multi-View Code Representation Tool for C and C++ Source Programs

Jaid Monwar Chowdhury[*], Ahmad Farhan Shahriar Chowdhury[†§], Humayra Binte Monwar[‡§], Mahmuda Naznin[†]
[*]*Dept. of Computer Science, University of Texas at Arlington, Arlington, TX, USA*
jxc3648@mavs.uta.edu
[†]*Dept. of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh*
{1905081@ugrad.cse.buet.ac.bd, mahmudanaznin@cse.buet.ac.bd}
[‡]*Dept. of Electrical and Electronic Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh*
2006159@eee.buet.ac.bd
[§]Equal contribution

***Abstract*—Multi-view code graphs that align abstract syntax trees, control flow graphs, and data flow graphs are now central to machine-learning models for software engineering. For C and C++, no single tool produces these aligned views without a complete build. We present ATLAS, a command-line tool that takes one or more C or C++ source files and emits an AST, a source-level inter-procedural CFG, a reaching-definition DFG, or any combination. The output is available as JSON, DOT, or PNG. ATLAS runs directly on source code and accepts partial input such as a project with missing headers, so it needs no compilation or build database. All views share one node namespace, so a downstream consumer can recover any single view by filtering edges. Command-line flags select views, collapse variables, and blacklist node categories to resize the emitted graph. On the TheAlgorithms corpora, ATLAS produces a correct CFG for 96.80% of C files and 91.67% of C++ files, including files that do not compile. It already serves as the CFG front-end of an LLM-based unit-test generation framework for C. ATLAS is open source and ships as a Docker image with a screencast walkthrough. Demo link: https://youtu.be/50DvEbenp14.**

***Index Terms*—Multi-view code representation, Abstract syntax tree, Control flow graph, Data flow graph, C/C++, ML4SE.**

I. Introduction

Software maintenance and machine learning for software engineering (ML4SE) tasks increasingly rely on source-level program views that expose syntax, control flow, and data flow. Multi-view code graphs align abstract syntax trees (ASTs) [1], control flow graphs (CFGs) [2], and data flow graphs (DFGs) [3] to capture syntactic structure, execution paths, and variable dependencies. These graphs encode the syntactic structure, execution paths, and variable dependencies that text-based models cannot observe directly. They support tasks such as test generation, code search, and vulnerability detection [4]–[8]. Works like GraphCodeBERT [9] and CodeSAM [10] show that models using structural views consistently outperform text-based baselines. Yet for C/C++, to our knowledge, no single existing tool produces these aligned graphs without a complete build environment.

Most available tools address only parts of this problem. They are (a) restricted to compiled code and operate on intermediate representations (IRs) that discard source-level variable names [11], (b) limited to fixed non-configurable graphs or syntax parsing with no control or data flow [12], [13], and (c) designed for memory-managed languages only [14]. Extending existing multi-view frameworks to C and C++ requires reimplementing core analysis, because pointer aliasing, manual memory management, and the absence of runtime type information (RTTI) each break assumptions those frameworks rely on [15]. Practitioners must therefore combine several single-purpose tools to obtain all three views.

We present ATLAS, a command-line tool for multi-view code representation of C and C++ that takes a single source file or a multi-file project and emits an AST, a source-level inter-procedural CFG, a reaching-definition DFG, or any combination, in JSON, DOT, or PNG. It models data flow through references and aggregate members without whole-program points-to analysis. We make the following contributions.

- Build-free extraction that runs on partial inputs such as a submodule with missing headers, without requiring a build or compilation database.
- Cross-view node alignment through shared identifiers, which lets users recover any single view by filtering edges, with no re-analysis.
- Three CLI flags, view selection, variable collapsing, and node blacklisting, that resize the emitted graph without re-running the analysis.
- Empirical evidence of utility on real C and C++ corpora and as the input to a downstream test-generation system.

On the TheAlgorithms corpora, ATLAS produces a correct CFG for 96.80% of C files and 91.67% of C++ files, including files that cannot be compiled due to missing dependencies. As downstream evidence, ATLAS has been adopted as the CFG front-end of SPARC [16], an LLM-based unit-test generation framework in C, where it raises test coverage over a no-context baseline (Section V-C). ATLAS lowers the barrier for ML4SE researchers who need aligned multi-view graphs for C/C++.

II. Related Tools

Compiler-based frameworks such as SVF [17] and PhASAR [11], which operate on LLVM [18] IR, provide precise analysis but require a resolved build environment. They also operate on IRs that discard source-level variable

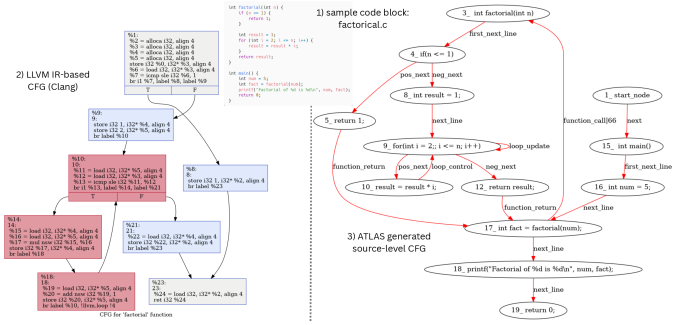

Fig. 1. Comparison of CFG representations for the same C program: Clang/LLVM IR (left) versus ATLAS source-level CFG (right).

TABLE I
FEATURE COMPARISON WITH RELATED MULTI-VIEW AND STATIC ANALYSIS TOOLS FOR C/C++

Tool	Build req.	Source-level CFG	Inter-proc. CFG	DFG	Multi-view aligned	Partial input
ATLAS	×	✓	✓	✓	yes (shared ids)	✓
Joern	×	✓	partial	✓	CPG (fixed)	✓
CodeQL	✓	×	✓	✓	×	×
SVF / PhASAR	yes (LLVM IR)	×	✓	✓	×	×
Clang Static Analyzer	✓	✓	partial	partial	×	×
Tree-sitter	×	×	×	×	×	✓
COMEX (Java / C#)	✓	✓	partial	✓	✓	×

names and structural context, which downstream models need in order to map results back to code (Figure 1). Query-based analyzers such as CodeQL [19] and the Clang Static Analyzer [20] share the same build requirement.

In contrast, Joern [12] avoids the build requirement and supports C/C++. However, it provides no means to select which views to extract or to combine them into aligned multi-view representations. Its analysis is also primarily intra-procedural, so data flow that crosses function boundaries is not captured.

Similarly, Tree-sitter [13] parses source code without compilation, but produces a syntax tree only. Without control or data flow, a model built on its output cannot reason about execution paths or variable dependencies, which are required for program analysis tasks.

Additionally, COMEX [14] produces configurable multi-view graphs with shared node identifiers directly from source, but targets memory-managed languages only, and its core analysis relies on properties that do not hold in C/C++. Three C/C++ semantics break those properties. Pointer aliasing creates ambiguous data flow, manual memory management removes the safety guarantees that reachability analysis depends on, and the absence of RTTI leaves virtual and function-pointer call targets unresolved [15]. Extending an existing multi-view framework to C/C++ therefore requires rebuilding its core analysis rather than modifying it.

To our knowledge, ATLAS is the only tool that satisfies all three requirements. Table I summarizes the comparison, where ATLAS is the only listed tool that combines source-level operation, inter-procedural CFG construction, partial-input tolerance, and shared-identifier multi-view alignment.

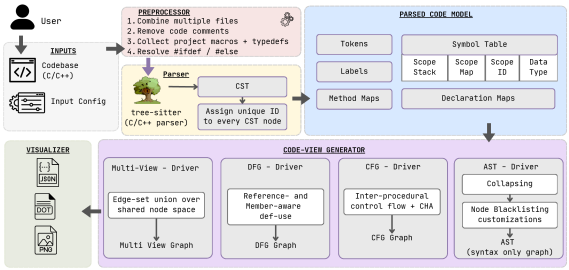

Fig. 2. Overview of the ATLAS four-stage pipeline: Preprocessor, Code-View Generator (AST, CFG, DFG, Multi-View drivers), and Visualizer.

III. THE ATLAS TOOL

ATLAS is a command-line interface (CLI) that takes one or more C/C++ source files and emits a set of aligned graph representations, working on source-level syntax trees rather than a compiler IR. Its parser accepts any syntactically valid C/C++ regardless of whether the referenced types, functions, or templates are defined in the analyzed files, so it can analyze extracted submodules, auto-generated code, and incomplete codebases that do not build.

ATLAS runs a four-stage pipeline (Figure 2): preprocessor, parser, code-view generator, and visualizer. It is invoked in one command, e.g. `atlas --lang c|cpp --code-folder PATH --graphs ast,cfg,dfg --output all`.

A. System Architecture

Preprocessor. The preprocessor merges a multi-file project into one file, collecting project-local macros and `typedefs` and inlining intra-project headers while keeping system headers as opaque declarations. For conditional compilation, C keeps both `#ifdef`/`#else` arms and C++ takes the arm selected by in-source `#define` values.

Parser. The parser runs Tree-sitter [13] over the merged file to build a syntax tree, a symbol table, and a unique integer identifier for every node. Symbols of unknown type are tagged `unknown_t` and treated as wildcards, so they do not block call-graph and def–use resolution. Because all drivers add edges over the shared identifiers, the AST, CFG, and DFG share one node namespace.

Code-View Generator. Four drivers (AST, CFG, DFG, Multi-View) operate over this model in the shared identifier space. Two options apply to every driver. Variable collapsing merges repeated identifiers into one node, and node blacklisting drops all nodes of a chosen syntactic category. Both reduce graph size before training.

Visualizer. Each view is exported as JSON for machines, Graphviz [21] DOT for other tools, and PNG for visual inspection. Multiple requested views are merged into one graph with edges colored by view class.

B. Code Views

AST. The AST driver prunes the Concrete Syntax Tree (CST) of nodes with no semantic content, leaving a syntax-only graph whose nodes carry a grammar category and source

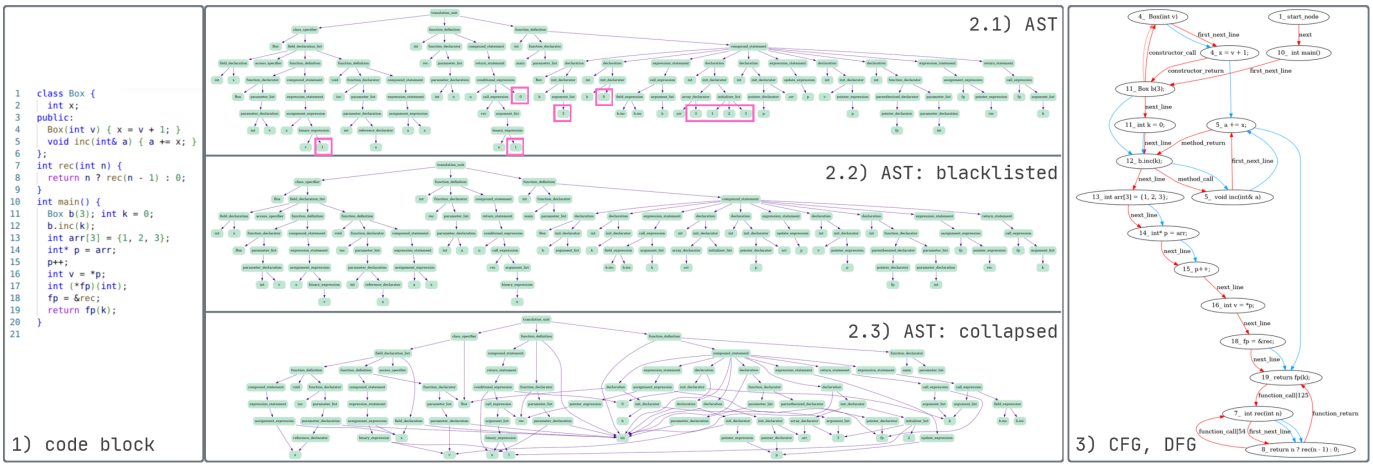

Fig. 3. Visual representations of code analysis structures by ATLAS: **1)** Sample C++ code block. **2.1)** Default AST showing the full hierarchical syntactic structure of the source code (123 nodes, 122 edges). **2.2)** Blacklisted AST with numeric literals removed, reducing noise while preserving structural and identifier nodes (114 nodes, 113 edges). Highlighted nodes in (2.1) are the numeric literals absent from (2.2). **2.3)** Collapsed AST with repeated occurrences of the same identifier merged into a single node, reducing redundancy (89 nodes, 122 edges). **3)** Combined CFG+DFG overlaying labeled control-flow edges (red) and def–use data-flow edges (blue) on a shared set of source-statement nodes.

range. It is the view on which variable collapsing and node blacklisting are most often applied. Figure 3(2.1–2.3) shows their effect on the running example. The default AST has 123 nodes (2.1). Passing `--blacklisted number_literal` prunes the numeric-literal leaves to 114 nodes (2.2), and `--collapsed` merges each repeated variable into one node for 89 nodes (2.3).

CFG. The CFG driver builds a source statement-level, inter-procedural control-flow graph directly over the source, so each node keeps its source-level name and structure with no lowering or basic-block grouping. It resolves direct calls, virtual calls through Class Hierarchy Analysis (CHA) [22], and function-pointer calls through the pointer's last visible assignment, fanning out to signature-compatible callees when a target is unresolved (Section V-B).

DFG. The DFG driver adds reaching-definition [23] edges at the same statement granularity, with two inter-procedural flows that plain def–use tracking would miss. A pass-by-reference argument shares its parameter's definition so callee modifications reach the caller, and a member access keeps the object's identity across calls. Aliasing through pointer arithmetic, heap-stored pointers, or `void*` callbacks is not modeled, since ATLAS has no build and thus no points-to graph.

Multi-View. Because identifiers are shared, a multi-view output is just the union of the selected views' edges over one node set, for any non-empty subset of {AST, CFG, DFG}. Figure 3(3) shows the combined CFG and DFG for the running example, with control-flow edges in red and def–use edges in blue over one shared set of source-statement nodes.

IV. Demonstration

A. Installation and Invocation

ATLAS ships as a Docker image. A single command builds it, and no system-wide toolchain or build database is required.

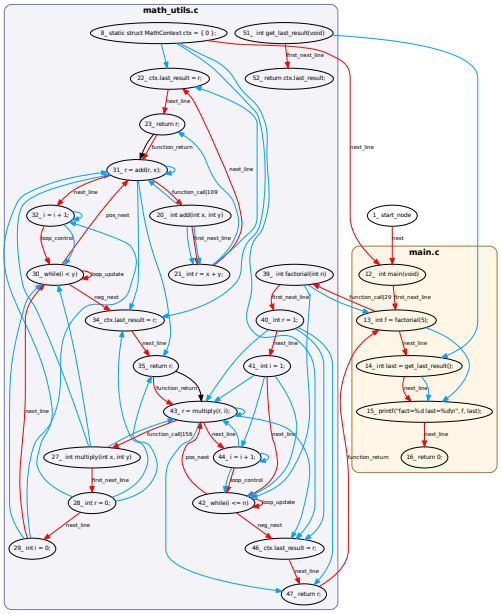

Fig. 4. Combined CFG+DFG ATLAS produces for the three-file `math_project` with `math_utils.h` deleted. Red edges are inter-file calls (labeled `function_call|`*`lineno`*), blue edges inter-file def–use on the file-scope `ctx`. The missing header's `MathContext` type is tagged `unknown_t` but still does not block edge construction (Section III-A).

```
$ docker build -t atlas .
$ atlas --lang c --code-folder PATH \
        --graphs ast,cfg,dfg --output all
```

B. Scenario: Analyzing a Project with a Missing Header

This section walks through a single end-to-end use of ATLAS on a multi-file C project to show that ATLAS produces

a connected inter-procedural CFG and DFG *across source files* even when the header declaring the shared interface is missing from disk. The header `math_utils.h` declares the functions `add`, `multiply`, `factorial`, and `get_last_result`, together with a small struct `MathContext { int last_result; }`. The `math_utils.c` implements the four functions and writes each computed result to a file-scope `static struct MathContext ctx`. `main.c` calls `factorial(5)`, then reads `ctx.last_result` indirectly through `get_last_result()`. We remove `math_utils.h` from disk before invocation.

The input is the three-file project `math_project/`, comprising `math_utils.h` (deleted), `math_utils.c` (35 lines), and `main.c` (8 lines). A compiler-based pipeline cannot proceed on this input: the `#include "math_utils.h"` directive on line 1 of `math_utils.c` fails to locate the header, and parsing halts before any analysis can run. ATLAS proceeds because its parser admits any syntactically-valid C without resolving `#included` declarations. The type `MathContext` referenced by the file-scope `ctx` declaration is recorded in the symbol table with an `unknown_t` tag (Section III-A), which downstream drivers treat as a wildcard at signature-matching and field-access sites:

```
$ clang -c math_utils.c
math_utils.c:1:10: fatal error:
    'math_utils.h' file not found

$ atlas --lang c \
        --code-folder math_project \
        --combined-name math_analysis \
        --graphs cfg,dfg --output all
[ok] output/math_analysis.{json,dot,png}
```

Figure 4 shows the resulting combined CFG+DFG. ATLAS itself emits a single merged graph. We group nodes into per-file clusters as a post-processing step on the DOT output. The inter-file call edge from `main`'s `int f = factorial(5);` into `factorial`'s entry node is constructed despite the missing declaration as signature matching against the visible definition in `math_utils.c` is sufficient. The inter-file def–use chain on the file-scope `ctx` variable is constructed for the same reason as the field `ctx.last_result` is written inside `add`, `multiply`, and `factorial`, and read from `get_last_result()`, which `main` calls. Each of these edges crosses the cluster boundary in Figure 4, and each carries the same node identifier scheme described in Section III-A. ATLAS also emits the same graph as node-link JSON.

Restoring `math_utils.h` and re-running ATLAS produces a structurally-equivalent graph, differing only by the single `declaration` node the header itself contributes (29 nodes / 74 edges without it, 30 nodes / 75 edges with). The partial-input graph is therefore a subset of the full graph, not an approximation of it. The same `--collapsed` and `--blacklisted` flags resize the emitted graph without re-running the analysis, letting a downstream model trade structural detail for graph size (Figure 3).

V. Evaluation

We evaluate ATLAS along three dimensions. First, the fraction of a representative C and C++ corpus on which it produces correct output. Second, the scope of its inter-procedural call resolution and its runtime cost on production codebases in both languages. Third, the utility of its output as input to a downstream ML4SE task.

A. Coverage of Language Constructs

We applied ATLAS to the `TheAlgorithms` C and C++ datasets [24], which contain 406 C and 360 C++ files. ATLAS produces a non-empty CFG and DFG for every syntactically valid input. To validate correctness, we fixed the property checklist before any inspection began, then had three of the authors independently check each emitted graph against its source. A graph counts as correct only when it passes every property on the checklist, namely matching function entries, branch and loop topology, switch fan-out, return edges, same-unit inter-procedural calls, and reaching-definition consistency for unambiguous def–use pairs. We reconcile the three independent judgments by majority vote. ATLAS produces a correct CFG for 393 C files (96.80%) and 330 C++ files (91.67%). It produces a correct DFG for 371 C files (91.38%) and 326 C++ files (90.56%). Each remaining file contains exactly one of five constructs the current implementation does not model (Table II). These are documented limitations of the support matrix, not implementation bugs.

TABLE II
UNSUPPORTED CONSTRUCTS IN THE THEALGORITHMS CORPUS. CFG FAILURES ARE A SUBSET OF DFG FAILURES, SINCE THE DFG DEPENDS ON CORRECTNESS OF THE CFG. THE PER-LANGUAGE TOTALS THEREFORE COUNT DISTINCT FILES (35 IN C, 34 IN C++) RATHER THAN THE SUM OF THE CFG AND DFG COLUMNS.

	C (n=406)		**C++** (n=360)	
Construct	**CFG**	**DFG**	**CFG**	**DFG**
`goto` statements	3	3	0	0
Multi-threading	10	10	6	6
Runtime pointer arithmetic	0	22	0	2
Operator overloading	0	0	24	24
Static class members	0	0	0	2
Total unsupported	**13**	**35**	**30**	**34**

B. Resolution and Runtime Cost on Production Codebases

We ran ATLAS on two production codebases chosen for heavy indirect dispatch. On LevelDB [25] (C++, roughly 10K LOC), it finds 389 virtual and indirect call sites, and 92% of them resolve to a single callee. On libcurl [26] (C, roughly 124K LOC), every same-unit direct call resolves correctly, and only 1.58% of call sites are left unconnected, the ones that dispatch through struct-stored function pointers (Section VI).

On the 20 largest C programs in the corpus (61 to 309 LOC), CFG generation takes a median of 6.89 s at 135 MB peak memory on a single workstation (i7-10700K, 32GB DDR4 RAM, Ubuntu 24.04 LTS). The full libcurl run completes in roughly 23 minutes at about 534 MB.

C. Downstream Use

ATLAS-generated CFGs serve as the structural front-end of SPARC [16], an LLM-based unit-test generation framework for automated unit-test generation in C. SPARC consumes source-level control-flow paths directly from ATLAS's CFG JSON output. On 59 real-world and TheAlgorithms C subjects it reports a 31.36% absolute gain in line coverage and 26.01% in branch coverage over a no-context prompting baseline, and matches or exceeds KLEE [6] on its more complex subjects (full results in [16]). As a result, ATLAS's output carries enough structural context for a downstream LLM-based consumer to use it directly.

VI. Limitations and Future Work

ATLAS performs no points-to analysis, so the DFG misses aliasing through pointer arithmetic, heap-stored pointers, and `void*` callbacks, resulting in the 1.58% of unconnected libcurl call sites in Section V-B. An Andersen-style analysis would close most of this gap and is on the roadmap. Without a full C++ frontend, ATLAS approximates templates, exceptions, and complex inheritance. Operator overloading, `goto`, and concurrency are out of scope (Table II). We have validated graph correctness only on small files (Section V-A), not at the 10^5-LOC scale. A large-project sweep and a user study of real-world adoption are our main future work.

VII. Availability and Reproducibility

ATLAS is released under the MIT License at https://github.com/jaid-monwar/ATLAS-multi-view-code-representation-tool which includes the Docker image, tool usage instructions, and paper artifacts. A 5-minute screencast is available at https://youtu.be/50DvEbenp14.

References

[1] Wikipedia contributors, "Abstract syntax tree," https://en.wikipedia.org/wiki/Abstract_syntax_tree, accessed: 2026-05-31.

[2] GeeksforGeeks, "Control flow graph (CFG) in software engineering," https://www.geeksforgeeks.org/software-engineering/software-engineering-control-flow-graph-cfg/, accessed: 2026-05-31.

[3] ScienceDirect, "Data flow graph," https://www.sciencedirect.com/topics/computer-science/data-flow-graph, accessed: 2026-05-31.

[4] S. Chimalakonda, D. Das, A. Mathai, S. Tamilselvam, and A. Kumar, "The landscape of source code representation learning in ai-driven software engineering tasks," in *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2023, pp. 342–343.

[5] C. Liu and R. Jabbarvand, "A tool for in-depth analysis of code execution reasoning of large language models," 2025. [Online]. Available: https://arxiv.org/abs/2501.18482

[6] C. Cadar, D. Dunbar, and D. Engler, "Klee: unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'08. USA: USENIX Association, 2008, p. 209–224.

[7] A. Fontes and G. Gay, "The integration of machine learning into automated test generation: A systematic mapping study," *Software Testing, Verification and Reliability*, vol. 33, no. 4, p. e1845, 2023.

[8] V. H. S. Durelli, R. S. Durelli, S. S. Borges, A. T. Endo, M. M. Eler, D. R. C. Dias, and M. P. Guimarães, "Machine learning applied to software testing: A systematic mapping study," *IEEE Transactions on Reliability*, vol. 68, no. 3, pp. 1189–1212, 2019.

[9] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.

[10] A. Mathai, K. Sedamaki, D. Das, N. S. Mathews, S. Tamilselvam, S. Chimalakonda, and A. Kumar, "Codesam: Source code representation learning by infusing self-attention with multi-code-view graphs," 2024. [Online]. Available: https://arxiv.org/abs/2411.14611

[11] P. D. Schubert, B. Hermann, and E. Bodden, "Phasar: An inter-procedural static analysis framework for c/c++," in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2019, pp. 393–410.

[12] "Joern - the bug hunter's workbench," https://joern.io/, accessed: 2023-11-17.

[13] M. Brunsfeld and Contributors, "Tree-sitter," https://tree-sitter.github.io/tree-sitter/, 2023, accessed: 2023-11-17.

[14] D. Das, N. S. Mathews, A. Mathai, S. Tamilselvam, K. Sedamaki, S. Chimalakonda, and A. Kumar, "Comex: a tool for generating customized source code representations," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2054–2057.

[15] P. D. Schubert, B. Hermann, and E. Bodden, "Lossless, Persisted Summarization of Static Callgraph, Points-To and Data-Flow Analysis," in *35th European Conference on Object-Oriented Programming (ECOOP 2021)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), A. Møller and M. Sridharan, Eds., vol. 194. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 2:1–2:31. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2021.2

[16] J. M. Chowdhury, C.-A. Fu, and R. Jabbarvand, "Sparc: Scenario planning and reasoning for automated c unit test generation," 2026. [Online]. Available: https://arxiv.org/abs/2602.16671

[17] Y. Sui and J. Xue, "Svf: interprocedural static value-flow analysis in llvm," in *Proceedings of the 25th International Conference on Compiler Construction*, ser. CC '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 265–266. [Online]. Available: https://doi.org/10.1145/2892208.2892235

[18] C. Lattner and V. S. Adve, "Llvm: A compilation framework for lifelong program analysis & transformation," *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, pp. 75–86, 2004.

[19] P. Avgustinov, O. de Moor, M. P. Jones, and M. Schäfer, "QL: Object-oriented Queries on Relational Data," in *30th European Conference on Object-Oriented Programming (ECOOP 2016)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), S. Krishnamurthi and B. S. Lerner, Eds., vol. 56. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016, pp. 2:1–2:25. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2016.2

[20] LLVM Project, "Clang static analyzer," https://clang-analyzer.llvm.org/, accessed: 2026-05-29.

[21] E. R. Gansner and S. C. North, "An open graph visualization system and its applications to software engineering," *Software: Practice and Experience*, vol. 30, no. 11, pp. 1203–1233, 2000. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/1097-024X%28200009%2930%3A11%3C1203%3A%3AAID-SPE338%3E3.0.CO%3B2-N

[22] J. Dean, D. Grove, and C. Chambers, "Optimization of object-oriented programs using static class hierarchy analysis," in *ECOOP'95 — Object-Oriented Programming, 9th European Conference, Åarhus, Denmark, August 7–11, 1995*, M. Tokoro and R. Pareschi, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1995, pp. 77–101.

[23] G. A. Kildall, "A unified approach to global program optimization," in *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL '73. New York, NY, USA: Association for Computing Machinery, 1973, p. 194–206. [Online]. Available: https://doi.org/10.1145/512927.512945

[24] TheAlgorithms, "TheAlgorithms: Collection of various algorithms in C and C++ for educational purposes," https://github.com/TheAlgorithms/C and https://github.com/TheAlgorithms/C-Plus-Plus, 2025, GitHub repositories, accessed 17 November 2025.

[25] S. Ghemawat and J. Dean, "Leveldb," https://github.com/google/leveldb, gitHub repository, accessed 29 May 2026.

[26] D. Stenberg and curl contributors, "curl," https://github.com/curl/curl, gitHub repository, accessed 29 May 2026.